Développement d'un site web dynamique (UAA12)

Exercices : échange de données (GET/POST)

Objets d'apprentissage :

- ✔ Elaborer un formulaire sur la base d'une structure donnée.
- ✔ Construire une page WEB dynamique à l'aide du langage PHP.
- ✔ Construire une page WEB dynamique à l'aide du langage Javascript.
- ✔ Concevoir un formulaire répondant aux exigences d'un cahier des charges.
- ✔ Dynamiser un site WEB exclusivement à l'aide du langage Javascript.
- ✔ Associer des balises HTML de formulaires à leur sémantique.

Jusqu'à présent, nous avons vu qu'un script PHP peut être accédé en spécifiant le chemin de sa localisation comme URL sur un navigateur WEB. Par exemple, pour tester ton script, tu accèdes à l'url http://localhost/projets/mon_script.php. Lorsqu'on accède à une ressource sur internet via une URL, il est possible de spécifier des informations supplémentaires :

- soit via la **méthode GET** (directement dans l'URL)

On peut spécifier une information :

```
http://localhost/projets/recherche.php?req=informatique
```

Ou même plusieurs !

```
http://localhost/projets/recherche.php?req=informatique&jour=lundi
```

- soit via la **méthode POST** (via un formulaire HTML)

```
<form action="/recherche.php" method="POST">
  <label for="req">Votre recherche :</label>
  <input type="text" id="req" name="req">
</form>
```

Du coté PHP, on peut réceptionner les données envoyées via GET en utilisant la variable **$_GET** qui représente un tableau associatif (paires clé / valeur). Si on reprend l'exemple précédent, ce tableau contient :

Clé	*Valeur*
req	informatique
jour	lundi

On accède à une valeur particulière de ce tableau en précisant la clé (comme en JavaScript) :

```
<?php
if (isset($_GET)){
  if (isset($_GET['req'])){
    echo 'Recherche : ';
    echo $_GET['req'];
    echo '<br>';
  }
  else
    echo 'Pas de recherche spécifiée !';
  if (isset($_GET['jour'])){
    echo 'Jour de la recherche : ';
    echo $_GET['jour'];
  }
  else
    echo 'Jour inconnu !';
}
?>
```

Exercice 1 : Commande (GET)

Etape 1.

Réalise une page en HTML (`choix.html`) sur laquelle se trouve le formulaire suivant :



Etape 2.

Ajoute du code JavaScript à `choix.html` de telle sorte que, lorsqu'on clique sur le bouton « Commander », l'utilisateur soit redirigé vers une page PHP (`commande.php`) avec un argument GET (`article=...`) indiquant le nom de l'article que le client souhaite commander.

Pour rappel, l'instruction JS pour rediriger l'utilisateur vers une autre URL est :

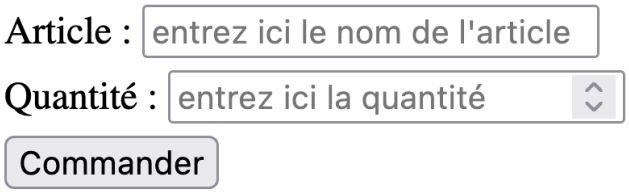
```
location.href="nouvelleURL";
```

Etape 3.

Réalise la page `commande.php` qui se charge d'afficher la confirmation de commande sous la forme « Vous venez de commander l'article '...' ».

Etape 4.

Ajoute le champ « Quantité » sur la page `choix.html`.



Fais également en sorte que le message de la page `commande.php` affiche maintenant « Vous venez de commander l'article '...' en X exemplaires ».

Exercice 2 : Créateur de playlist musicale (GET)

Étape 1

Crée une page HTML (`playlist.html`) contenant un formulaire avec :

- Une liste déroulante permettant de choisir un genre musical (Pop, Rock, Hip-Hop, Électro, K-Pop)
- Une liste déroulante pour choisir l'humeur (Énergique, Calme, Motivante, Mélancolique)
- Un bouton "Générer ma playlist"

Étape 2

Ajoute du code JavaScript qui, au clic sur le bouton, redirige l'utilisateur vers `generer_playlist.php` en passant les paramètres `genre` et `humeur` via la méthode GET.

Étape 3

Dans `generer_playlist.php`, affiche un message personnalisé du type : "Ta playlist **[genre]** pour une ambiance **[humeur]** est prête !"

Étape 4

Ajoute un champ numérique dans le formulaire pour demander combien de chansons l'utilisateur souhaite dans sa playlist (entre 5 et 20).

Modifie `generer_playlist.php` pour afficher : « Ta playlist contient **X** chansons de **[genre]** pour une ambiance **[humeur]** ».

Exercice 3 : Quizz (GET)

L'objectif de cet exercice est de créer un site WEB éducatif permettant de tester ses connaissances en répondant à des quizz comportant plusieurs questions à choix multiples. Pour le rendre attractif, ce site présentera les réponses sous forme de boutons qui se déplacent sur l'écran.

Du coté JavaScript, tu te chargeras de programmer les « boutons volants ». La réponse sélectionnée par l'utilisateur sera ensuite envoyée au serveur PHP en utilisant la méthode GET.

Du coté PHP, tu te chargeras de vérifier si la réponse est correcte ou pas.

Étape 1

Du coté HTML, crée une page comportant :

- un paragraphe
- suivi d'un div vide de 600px de large sur 400px de haut
- lui-même suivi d'un paragraphe indiquant « Cliquez sur la réponse correcte ! »

Prévois un identifiant pour le premier paragraphe et pour le div. Utilise les règles CSS pour donner une bordure au div, pour préciser ses dimensions et pour le mettre en position *relative* (nécessaire pour déplacer les boutons à l'intérieur).

Étape 2

Ajoute un script JavaScript qui se déclenchera une fois la page chargée.

Étape 3

Ajoute au script JavaScript les instructions nécessaires pour insérer quatre boutons au div.

Voici les étapes à suivre pour chaque bouton :

1. crée l'élément `<button>` via `document.createElement`[1]
2. ajoute-lui la classe CSS `boutonVolant` via la propriété `className`[2]
3. ajoute-lui un identifiant via la propriété `id`[3]
4. ajoute l'élément dans le div via `appendChild`

Modifie la définition de la classe CSS `boutonVolant` pour qu'elle impose une largeur minimale de 60px et une hauteur minimale de 30px. Fais également en sorte que les boutons aient une position absolue (`position:absolute`).

Observe le résultat : les boutons doivent être tous superposés dans le coin supérieur gauche du div.

1 https://www.w3schools.com/jsref/met_document_createelement.asp
2 https://www.w3schools.com/jsref/prop_html_classname.asp
3 https://www.w3schools.com/jsref/prop_html_id.asp

Étape 4

Occupe-toi maintenant du positionnement des boutons.

La position d'un bouton est définie par deux coordonnées `(posX,posY)` et son déplacement est déterminé par deux vitesses `(vitesseX,vitesseY)`.

Les valeurs des coordonnées et de la vitesse seront générées aléatoirement lors de leur création :

- La position en x du coin supérieur gauche du bouton sera un entier déterminé aléatoirement et compris entre `0` et `600-clientWidth` (étant donné que le div fait 600px de large, il faut soustraire la largeur du bouton pour éviter qu'il ne déborde en dehors du div)
- La position en y du coin supérieur gauche du bouton sera un entier déterminé aléatoirement et compris entre `0` et `400-clientHeight` (étant donné que le div fait 400px de hauteur, il faut soustraire la hauteur du bouton pour éviter qu'il ne déborde en dehors du div)
- Les vitesses en x et y seront un entier déterminé aléatoirement et compris entre `-5` et `5`

Voici l'instruction permettant de générer un nombre aléatoire entre deux entiers min et max (inclus) :

```
Math.floor(Math.random() * (max - min + 1)) + min;
```

Les coordonnées et la vitesse de chaque bouton seront enregistrées dans un tableau à deux dimensions. La clé est l'identifiant du bouton et les valeurs sont ses coordonnées et sa vitesse.

Voici un exemple du contenu de ce tableau 2D :

	posX	posY	vitesseX	vitesseY
1	30	50	2	-1
2	20	10	1	1
3	50	50	-5	2

Voici quelques instructions utile pour mettre en place ton tableau 2D :

```
// crée un tableau vide
let tabPos = [];
// crée un tableau vide pour la clé "1" (ce qui donne un tableau 2D)
tabPos[1] = [];
// ajoute la valeur 30 pour la clé "posX" dans le tableau vide de la clé
"1"
tabPos[1]["posX"] = 30;
// ajoute la valeur 50 pour la clé "posY" dans le tableau vide de la clé
"1"
tabPos[1]["posY"] = 50;
```

Pour positionner précisément un bouton à l'intérieur d'un div, il faut indiquer la position via les propriétés `style.left` et `style.top` :

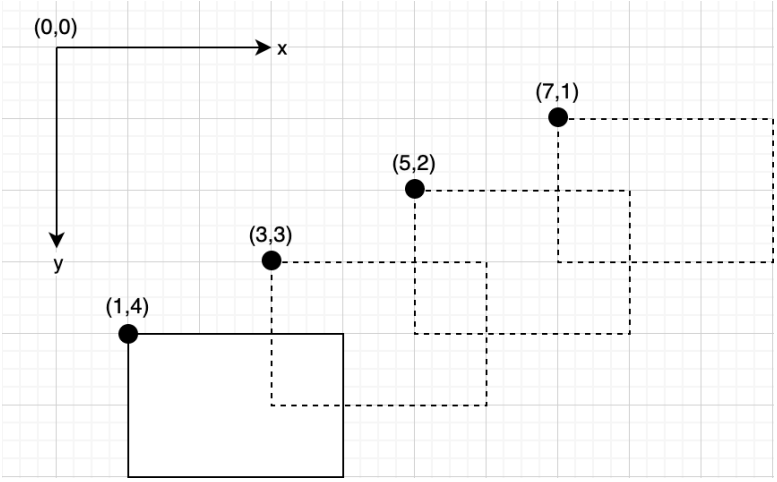
```
bouton.style.top = <valeur> + "px"
bouton.style.left = <valeur> + "px"
```

Attention : <valeur> est à récupérer directement depuis le tableaux des positions !

Étape 5

Occupe-toi maintenant du déplacement des boutons.

Le mouvement est simulé ainsi : si, à un moment, le bouton est positionné en (1,4) et a une vitesse de (2,-1), le bouton se déplacera de 2px vers la droite et de 1px vers le haut ; il passera alors aux positions (3,3), (5,2), (7,1), etc.



Crée la fonction `déplacerBoutons()` qui se charge de déplacer tous les boutons présents sur l'interface :

1. Cible l'ensemble des boutons grâce à `document.getElementsByClassName(classe)`. Attention, puisque cette méthode renvoie un tableau d'éléments, tu dois parcourir l'ensemble du tableau (à l'aide d'un **`for`**)
2. Pour chaque bouton :
   a) extrait sa position `(posX,posY)` et sa vitesse `(vitesseX,vitesseY)` depuis le tableau 2D grâce à son `id`
   b) calcule sa nouvelle position selon sa vitesse
   c) effectue le mécanisme de rebond (dans le cas où le bouton sortirait du div)
   d) enregistre sa nouvelle position dans le tableau des positions (`tabPos`)

Algorithme du mécanisme de rebond

```
    *
nouvellePosX = posX + vitesseX
nouvellePosY = posY + vitesseY
// Cas où le bouton sort à gauche ou à droite
    if (nouvellePosX < 0 OR nouvellePosX + largeur > 600)
 vitesseX = vitesseX * -1
 nouvellePosX = posX + vitesseX

// Cas où le bouton sort en haut ou en bas
    if (nouvellePosY < 0 OR nouvellePosY + hauteur > 400)
 vitesseY = vitesseY * -1
 nouvellePosY = posY + vitesseY
```

Etape 6

Fais en sorte que la fonction `déplacerBoutons()` soit exécutée toutes les 30 millisecondes. Utilise pour cela la fonction `setInterval(...)`[4].

Etape 7

Il est temps maintenant de s'occuper du premier paragraphe (celui qui affiche la question).

Génère tout d'abord deux entiers (`nombre1` et `nombre2`) entre 0 et 10 et affiche le texte « Que vaut <nombre1> x <nombre2> ? »

Etape 8

Fais en sorte que chacun des boutons comporte une réponse différente, avec le texte « <nombre1> x <nombre2> = <reponse> » avec :

Bouton 1 : `reponse = nombre1 * nombre2` (c'est-à-dire la bonne réponse)

Bouton 2 : `reponse = nombre1 + nombre2`

Bouton 3 : `reponse = (nombre1 * nombre2) + 2`

Bouton 4 : `reponse = (nombre1 * nombre2) + 6`

Indice : utilise la propriété `innerHTML` des boutons.

Fais en sorte que, lorsqu'on clique sur chacun de ces boutons, l'utilisateur soit redirigé vers la page `verification.php` en passant via la méthode GET les arguments suivants :

- `n1 = nombre1`
- `n2 = nombre2`
- `rep = reponse`

Indice : utilise la propriété `onclick` des boutons[5].

Etape 9

Crée un fichier `verification.php` qui :

- affiche « Vous ne pouvez pas accéder à cette page » si aucune donnée n'est transmise en GET
- affiche « Réponse correcte ! » si l'utilisateur a choisi la bonne réponse
- affiche « Réponse incorrecte ! » sinon

Etape 10 (dépassement)

Ajoute des boutons qui se déplacent aléatoirement dans l'interface (c'est à dire qui ont une vitesse qui change à chaque déplacement).

4 https://www.w3schools.com/jsref/met_win_setinterval.asp
5 https://stackoverflow.com/questions/15097315/change-onclick-attribute-with-javascript

Exercice 4 : Générateur de profil de streamer (POST)

Étape 1

Crée un formulaire HTML (`profil_streamer.html`) avec méthode POST contenant trois sections (`<fieldset>`) :

Section 1 : Identification

- Champ texte pour le pseudo (obligatoire)
- Champ email

Section 2 : Préférences de stream

- Boutons radio pour choisir la plateforme (Twitch, YouTube, TikTok)
- Cases à cocher pour les types de contenu (Gaming, Just Chatting, Créatif, Sport, Musique)
- Liste déroulante pour les heures de stream préférées (Matin, Après-midi, Soir, Nuit)

Section 3 : Statistiques

- Champ numérique pour le nombre d'abonnés (min: 0, max: 1000000)
- Champ numérique pour les heures de stream par semaine (min: 1, max: 100)

Ajoute un bouton de soumission "Créer mon profil".

Étape 2

Crée `afficher_profil.php` qui récupère les données POST et utilise `var_dump($_POST)` pour afficher toutes les données reçues.

Étape 3

Supprime le `var_dump()`. Affiche maintenant un profil personnalisé :

```
                    Bienvenue [pseudo] !
Voici ton profil de streamer :
- Plateforme : [plateforme]
- Types de contenu : [liste des types cochés]
- Horaires favoris : [horaire]
- Statistiques : [X] abonnés, [Y] heures/semaine
```

Étape 4

Calcule et affiche des statistiques supplémentaires :

- Nombre moyen d'abonnés par heure de stream par semaine
- Un message personnalisé selon le nombre d'abonnés :
    - < 100 : "Tu débutes, continue comme ça !"
    - 100-1000 : "Bon début, tu progresses !"
    - 1000-10000 : "Très bien, tu te développes !"
    - > 10000 : "Incroyable, tu es une star !"

Étape 5

Ajoute les validations HTML (`required`, `min`, `max`) pour empêcher l'envoi de données invalides.

Étape 6

Dans le fichier PHP, ajoute des vérifications côté serveur avec `isset()` pour t'assurer que toutes les données nécessaires sont présentes. Si des données manquent, affiche "Erreur : données incomplètes".

Étape 7 (dépassement)

Affiche une liste d'erreurs détaillées dans un cadre rouge si :

- Le pseudo est vide : "Le pseudo est obligatoire"
- Le nombre d'abonnés est négatif : "Le nombre d'abonnés doit être positif"
- Les heures de stream dépassent 168 (heures dans une semaine) : "Impossible de streamer plus de 168h/semaine !"

Exercice 5 : Création de compte gamer avec validation (POST + GET)

Contexte

Tu vas créer un système d'inscription pour une plateforme de gaming. Le formulaire sera sur la même page que le traitement PHP. En cas d'erreur, la page se rechargera elle-même en affichant un message d'erreur personnalisé sous le formulaire grâce à un paramètre GET.

Étape 1 : Mise en place du formulaire

Crée un fichier `inscription.php` qui contient un formulaire HTML avec la méthode POST. Ce formulaire doit pointer vers lui-même (utilise `action="inscription.php"`).

Le formulaire doit contenir :

- Un champ texte pour le pseudo (attribut `name="pseudo"`)
- Un champ email (attribut `name="email"`)
- Un champ numérique pour l'âge (attribut `name="age"`)
- Une liste déroulante pour choisir son jeu préféré avec 4 options (attribut `name="jeu"`)
- Un bouton de soumission

N'oublie pas d'ajouter des balises `<label>` pour chaque champ.

Étape 2 : Affichage conditionnel du message d'erreur

Au-dessus du formulaire, ajoute une `<div id="zone-erreur"></div>`.

Dans la partie PHP en haut de ton fichier `inscription.php`, vérifie si un paramètre GET nommé `erreur` existe. Utilise la fonction `isset()` pour cela.

Si ce paramètre existe, affiche un message d'erreur personnalisé dans la div selon la valeur du paramètre :

- Si `$_GET['erreur']` vaut `"pseudo_vide"`, affiche "Le pseudo est obligatoire"
- Si `$_GET['erreur']` vaut `"email_invalide"`, affiche "L'email n'est pas valide"
- Si `$_GET['erreur']` vaut `"age_insuffisant"`, affiche "Tu dois avoir au moins 13 ans"
- Si `$_GET['erreur']` vaut `"jeu_invalide"`, affiche "Tu dois choisir un jeu"

Applique du style CSS pour que cette zone d'erreur ait un fond rouge et une bordure.

Étape 3 : Vérification des données POST

Dans la même page `inscription.php`, ajoute du code PHP qui vérifie si des données ont été envoyées via POST.

Utilise `isset($_POST['pseudo'])` pour détecter si le formulaire a été soumis.

Si le formulaire a été soumis, tu dois vérifier chaque donnée :

- Vérification du pseudo :
    - Vérifie si le pseudo est vide (utilise `empty()`)
    - Si c'est le cas, tu dois rediriger vers la même page avec le paramètre GET `erreur=pseudo_vide`
- Vérification de l'email :
    - Vérifie si l'email est valide (utilise `filter_var()`[6])
    - Si ce n'est pas valide, redirige avec `erreur=email_invalide`

Vérification de l'âge :

- Vérifie si l'âge est inférieur à 13
- Si oui, redirige avec `erreur=age_insuffisant`

Vérification du jeu :

- Vérifie si le jeu choisi fait partie des quatre jeux autorisés
- Si non, redirige avec `erreur=jeu_invalide`

Pour effectuer une redirection en PHP, utilise la fonction `header('Location: URL')` suivie de `exit;`.

Étape 4 : Redirection en cas de succès

Si toutes les vérifications sont passées (aucune erreur détectée), redirige l'utilisateur vers une page `succes.php`.

Tu peux passer le pseudo en paramètre GET pour l'afficher sur la page de succès :

```php
header('Location: succes.php?pseudo=' . urlencode($_POST['pseudo']));
```

Étape 5 : Création de la page de succès

Crée un fichier `succes.php` qui affiche un message de bienvenue.

Récupère le pseudo depuis le paramètre GET et affiche : "Bienvenue **[pseudo]** ! Ton compte a été créé avec succès."

Utilise `htmlspecialchars()` pour sécuriser l'affichage du pseudo.

Vérifie d'abord avec `isset()` que le paramètre `pseudo` existe bien dans GET.

6 https://www.w3schools.com/php/func_filter_var.asp

Étape 6 (dépassement): Conservation des données du formulaire

Actuellement, lorsqu'il y a une erreur et que la page se recharge, l'utilisateur doit tout re-saisir.

Modifie ton code pour passer également les données valides en GET lors de la redirection d'erreur.

Ensuite, dans la partie HTML de `inscription.php`, fais en sorte que les champs se remplissent automatiquement si ces paramètres GET existent.

Étape 7 (dépassement) : Gestion de plusieurs erreurs simultanées

Actuellement, ton système ne gère qu'une seule erreur à la fois. Modifie ton code pour :

1. Créer un tableau `$erreurs = []` au lieu d'une simple variable
2. Ajouter chaque erreur détectée dans ce tableau au lieu de rediriger immédiatement
3. Si le tableau contient des erreurs, convertis-le en chaîne de caractères séparée par des virgules (utilise `implode()`[7])
4. Redirige avec ce paramètre : `erreur=pseudo_vide,email_invalide`
5. Dans la zone d'affichage des erreurs, sépare cette chaîne (utilise `explode()`[8]) et affiche tous les messages dans une liste `<ul>`

Points d'attention

- **Sécurité** : Les attributs HTML comme `required` ne sont pas suffisants car ils peuvent être modifiés côté client. La vérification PHP côté serveur est indispensable.
- **Redirection** : Assure-toi qu'il n'y a aucun affichage (pas même un espace) avant l'instruction `header()`, sinon la redirection échouera.

7 https://www.w3schools.com/php/func_string_implode.asp
8 https://www.w3schools.com/php/func_string_explode.asp

Exercice 6 : Mes anniversaires (POST)

L'objectif de cet exercice est de réaliser un formulaire permettant à l'utilisateur d'entrer son nom et sa date de naissance. Ces données sont envoyées au serveur et celui-ci produira une page listant ses futurs anniversaires et affichant un certain nombre d'informations choisies par l'utilisateur.

Etape 1

Crée un formulaire HTML comportant trois sections (balises `<fieldset>`) :

1. La première, nommée « Identification » (balise `<legend>`) contient simplement un champ textuel (balise `<input type="text">`) demandant le prénom de l'utilisateur
2. La seconde, nommée « Date de naissance », contient :
   i. une liste déroulante (balise `<select>`) proposant à l'utilisateur de sélectionner son jour de naissance (de 1 à 31)
   ii. des boutons radio (balise `<input type="radio">`) proposant à l'utilisateur de sélectionner son mois de naissance
   iii. un champ numérique (balise `<input type="number">`) proposant à l'utilisateur d'entrer son année de naissance
3. La troisième, nommée « Colonnes à afficher », contient trois cases à cocher (balise `<input type="checkbox">`) proposant à l'utilisateur de choisir s'il souhaite afficher les dates et/ou les jours et/ou l'âge

Ce formulaire comporte également un bouton de soumission (balise `<input type="submit">`) permettant d'envoyer les données du formulaire au serveur lorsque l'utilisateur clique dessus.

Quelques points d'attention :

- ✔ N'oublie pas d'ajouter des étiquettes aux champs (balise `<label>`)
- ✔ La date par défaut est le 1er janvier 2000
- ✔ Choisis la méthode d'envoi POST pour le formulaire ; fais en sorte que la balise `<form>` comporte les attributs suivants : `<form action="data.php" method="post">`

Etape 2

Crée un fichier nommé `data.php` qui se charge simplement d'afficher les données reçues en POST. Pour cela, tu peux ajouter l'instruction suivante dans ton script :

```php
<?php
var_dump($_POST);
?>
```

Remplis les données de ton choix dans le formulaire, clique sur le bouton, et observe le résultat. Comprends-tu l'importance de spécifier l'attribut `name` d'un champ ?

Etape 3

Supprime l'instruction précédente. Tu vas maintenant t'attaquer à l'affichage des futurs anniversaires.

Tout d'abord, fais en sorte que la page affiche « Chère / Cher `<prénom>`, voici vos futurs anniversaires. »

Ensuite, propose un tableau contenant plusieurs colonnes :

1. La première contient la liste des années allant de 2023 à 2050
2. Si l'utilisateur a coché l'option « date d'anniversaire », une colonne reprend chaque date des futurs anniversaires au format `JJ/MM/AAAA`
3. Si l'utilisateur a coché l'option « jours d'anniversaire », une colonne reprend chaque jour des futurs anniversaires (lundi, mardi, etc.)
4. Si l'utilisateur a coché l'option « âge », une colonne indique l'âge qu'il atteindra lors de chacun de ses futurs anniversaires.

Pour le calcul du jour (colonne 3), utilise la fonction `date(…)`. Voici un exemple d'utilisation :

```php
$date = "1998/12/23";
$jour = date("l", strtotime($date));
```

Dans cette exemple, la variable `$jour` contiendra « Wednesday ».

Etape 4

Le prénom est champ obligatoire et l'année de naissance doit être comprise entre 1900 et 2023 : ajoute les contraintes nécessaire en HTML pour faire en sorte que l'utilisateur ne puisse pas soumettre le formulaire si les données sont manquantes ou incorrectes. Utilise pour cela les attributs `required`, `min` et `max`.

Teste le mécanisme de vérification en tentant de soumettre le formulaire sans indiquer de prénom ou en indiquant une année de naissance invalide.

Etape 5

Ces attributs permettant de vérifier les données d'un formulaire sont-ils sécurisés ? Pour le savoir, rends-toi dans l'inspecteur web de ton navigateur, édite l'HTML en supprimant ces attributs et tente une nouvelle fois d'envoyer le formulaire…

Etape 6

Les attributs `required`, `min` et `max` sont des mécanismes de vérification basée sur le JavaScript : elles sont effectuées coté client. Dès lors, l'utilisateur peut à sa guise modifier le code pour outrepasser ces mécanismes.

Pour rendre ton projet plus sécurisé, il faut effectuer également la vérification du coté du serveur, c'est à dire en PHP !

À l'aide de la fonction `isset(…)`, fais en sorte que le page PHP n'affiche rien si certaines informations transmises ne sont pas correctes.

Etape 7 (dépassement)

Fais en sorte d'afficher le message d'erreur « Les erreurs suivantes doivent être corrigées :» suivi des messages suivants (en fonction des cas) dans une liste (balises `<ul>` et `<li>`):

- lorsque le prénom est manquant : « Le prénom est obligatoire. »
- lorsque l'année de naissance est trop grande : « L'année de naissance doit être avant 2023. »
- lorsque l'année de naissance est trop petite : « L'année de naissance doit être après 1900. »

Affiche la / les erreur(s) en gras dans un cadre rouge.

Références

Les présents exercices ont été élaborés à l'aide des ressources suivantes :

- Forbes, A. (2013). The Joy of PHP: A Beginner's Guide to Programming Interactive Web Sites.
- Vaswani, V. (2021). PHP A Beginner's Guide.
- Cours de « Développement d'applications WEB » (2018), Hénallux